

LLM-based Compatibility Detection

xxxxxxx[1]

1 Introduction

2 Background

2.1 API Compatibility Detection

2.2 API Compatibility Type

2.3 Motivating Example

3 Approach

This section details our methodology for detecting Python API incompatibilities using Large Language Models (LLMs) with structured Chain-of-Thought (COT) prompting. We first outline the overall workflow and core challenges, then elaborate on the design principles compared to existing methods, and technical implementation

3.1 CoT

3.2 Overview and Challenges

Our approach targets code-level changed API pairs (excluding module-level, class-level, and API addition/deletion changes) between two versions of a Python project. Figure x illustrates the end-to-end workflow. First, we extract all APIs from two versions of the project and filter API pairs to retain only those with code-level modifications. Then we instruct LLM with COT guidance to performs line-by-line code analysis. Finally, The LLM generates a structured JSON output, including detected incompatibility types, detailed reasons for each type, and auxiliary information.

figure x

During this three-step process, we need to address the following three challenges:

- **High-Level Description Misalignment (C1):** Existing LLM-based methods[x] often rely on high-level incompatibility descriptions rather than code-level semantics. This disconnect causes LLMs to struggle with mapping abstract descriptions to concrete code changes.
- **Scalability with Incompatibility Categories (C2):** As the number of incompatibility categories increases, naive prompting leads to exponential growth in prompt token size and LLM runtime. For N categories, this approach requires N independent LLM calls per API pair, making it infeasible for large-scale analysis.
- **Unreliable and Inconsistent Outputs (C3):** LLMs exhibit inherent output randomness—they may produce contradictory judgments for the same prompt (e.g., labeling a parameter rename as “compatible” in one run and “incompatible” in another) or include internally inconsistent reasoning (e.g., citing non-existent code lines to support a claim).
- **Unstructured COT Reasoning (C4):** Without guided thinking frameworks, LLMs’ intermediate reasoning (e.g., how they identify a parameter default value change) remains unstructured. This makes it impossible to verify the validity of the LLM’s conclusions.

3.3 Methodology Design

To address the above challenges, we design a one-pass COT analysis framework with structured prompting, built on insights from prior empirical studies and innovations in LLM guidance.

We first establish a comprehensive taxonomy of Python API incompatibilities, extending the 7-category framework proposed by Shen et al. [x] (a foundational empirical study on Python API evolution). Shen et al. identified incompatibilities rooted in parameters (CI1-CI5), exceptions (CI6), and return values (CI7). Through our own data collection, we validated these 7 categories and discovered two new, previously unreported types:

CI8: global variable change **CI9:** reference variable change

Table x

3.3.1 Limitation of Current Methods. Traditional static analysis methods (e.g., Shen et al. [2024]) rely on handcrafted rules. However, Rule design requires expert knowledge making it hard to generalize to new libraries or unknown incompatibilities. For example, shen define data structure P to record parameter information for the detection of CI1-CI5. However, without detailed study into incompatible examples. it is hard to design rules for detections. Besides, These methods fail to interpret semantic context while many crucial infomation should be analyzed through variable names, comments, usage in function bodies, etc. For example, Shen et al.’s method cannot recognize that renaming a parameter from arguments to args is a compatibility issue—since the AST pattern for parameter names differs, but the static rule does not link the semantic similarity between arguments and args.

In our method, by leveraging LLMs’ semantic understanding capabilities, our approach directly addresses this limitation. The LLM integrates contextual cues to identify semantic changes, even when syntactic patterns are non-trivial.

3.3.2 One-Pass COT Analysis Framework. To tackle scalability and output unreliability, we design a one-pass COT framework guided by two core principles: two-stage analysis and unified scanning. First, the LLMs is instructed to process the parameter list, which is syntactically structured, then analyze the function body, which is semantically rich—aligning with how developers naturally review code changes. We embed rules for all 9 incompatibility categories into a single prompt to enable the LLM to detect multiple incompatibilities in one analysis pass, avoiding repeated LLM calls. The framework is divided into four sequential phases, each with predefined data structures and rules to guide the LLM’s reasoning. This design offers three key advantages: efficiency, as runtime is independent of the number of incompatibility categories (all rules are processed in one pass); reduced misjudgment, as it enforces dependencies between interdependent categories (e.g., analyzing CI1 (parameter addition) before CI4 (default value change) to avoid misclassifying added parameters as default value changes); and structured reasoning, as each phase outputs intermediate data structures like parameter mappings and exception type lists, which make the LLM’s logic traceable and verifiable.

3.3.3 Structured Prompt Design. The prompt is the backbone of our framework, structured to enforce consistency and guide the LLM’s COT reasoning. It includes four components: Instructions, Analysis Phases, Source Code, and Examples. A simplified template is shown below (full rules are provided in Appendix A):

Analyze two versions of a Python API function (`code\_v1` and `code\_v2`) to detect incompatibilities

Instructions

Please strictly follow the `Analysis Phases` to identify all incompatibility types (CI1 -

- `additional\_info`: Key variables/structures used in analysis (e.g., parameter mappings).
- `reasons`: One-sentence explanations linking code changes to incompatibility types.
- `incompatibility`: List of detected types (e.g., ["CI3", "CI7"]).

Analysis Phases

Phase 0: Initialization

Initialize the result dictionary:

```
{
  "incompatibility": [],
  "reasons": [],
  "additional_info": {"param_mapping": {}, "exception_list_v1": [], ...}
}
```

Phase 1: Parameter List Processing

Extract parameter lists from `code\_v1` and `code\_v2`, then build:

- `P1`/`P2`: Dictionaries of parameters (keys: names, values: type/default/position).
- `param\_mapping`: Cross-version mapping of parameters (e.g., {"fname": "path"} for CI3).

Rules:

1. For positional parameters: Match by order; flag missing parameters as CI1.
2. For default parameters: Compare default values; flag changes as CI4.
- ... (rules for CI2-CI3, CI5)

Phase 2: Initial Incompatibility Analysis (CI1, CI4)

Use `P1`/`P2` to update results:

- If `P2` has more positional parameters than `P1`: Add "CI1" to `incompatibility`; reason: "Add
- If a default parameter's value differs between `P1` and `P2`: Add "CI4" to `incompatibility`;

Phase 3: Function Body Processing (Line-by-Line)

Initialize data structures for `code\_v1`/`code\_v2`:

- `E1`/`E2`: Lists of exception types thrown (e.g., ["ValueError", "TypeError"]).
- `G1`/`G2`: Global variables referenced (e.g., ["MAX_RETRIES"]).
- `R1`/`R2`: Return value types (e.g., ["NoneType", "DataFrame"]).

Process each line to populate these structures:

- For `raise` statements: Add exception type to `E1`/`E2`.
- For global variable assignments: Add variable name to `G1`/`G2`.
- ... (rules for CI6-CI9)

Phase 4: Comprehensive Incompatibility Analysis

```

Use Phase 1-3 data structures to detect remaining types:
- CI6: If `E1` != `E2` for the same error condition: Add "CI6"; reason: "Throws X instead
- CI8: If `G1` != `G2`: Add "CI8"; reason: "Global variable X modified in v2".
... (rules for CI7, CI9)

### Source Code
"code_v1":
def to_stata(self, fname, convert_dates=None, encoding='latin -1'):
    writer = statawriter(fname, ...)
    ...

"code_v2":
def to_stata(self, path, convert_dates=None):
    writer = statawriter(path, ...)
    ...

### Examples
Example 1 (CI1 + CI3):
Output: {
  "additional_info": {"param_mapping": {"fname": "path"}},
  "reasons": ["Deleted parameter 'encoding' (CI1)", "Renamed 'fname' to 'path' (CI3)"],
  "incompatibility": ["CI1", "CI3"]
}

```

3.3.4 Data Structure and Detection Design. The analysis leverages several core data structures to capture critical information from function signatures and bodies, facilitating subsequent incompatibility checks. Based on the detecting process for different incompatibility types, we design following data structure which is initialized at beginning and update throughout the scanning of the source code.

Parameter Lists (P1, P2): These lists encode the parameters of source code. Each parameter is represented as a tuple (name, position index, type, default value), where type includes positional, optional (with a default value), or kwargs, enabling direct comparison of parameter presence, order, and defaults.

KWARGS: A string storing the name of the kwargs parameter if present in both code v1 and code v2; empty otherwise. This structure anchors the analysis of keyword argument keys accessed within the function body.

ReName: A list of tuples (name1, name2): name1 (from code v1) and name2 (from code v2) are semantically similar or share the same position index, indicating potential parameter renaming.

Keyword Argument Keys (K1, K2): Lists of keys explicitly accessed via kwargs (e.g., kwargs.get('key') or kwargs['key']) in code v1 and code v2, respectively. These capture dependencies on dynamic keyword arguments.

Renamed Parameter Usage (ReNUsage1, ReNUsage2): Dictionaries mapping parameters in ReName pairs to lists of code lines where they are used. This tracks contextual usage to validate renaming.

Condition Statements (C1, C2): Lists of condition strings (e.g., `if x > 0`) extracted from code v1 and code v2, indexed sequentially to link with dependent logic (e.g., assignments, exceptions).

Data Flow (D1, D2): Lists of tuples (variable name, variable type, dependent variables, condition indices) capturing variable assignments. variable type distinguishes local variable, global variable, and reference variable (e.g., `self`), while dependent variables and condition indices track dependencies and associated conditions.

Exceptions (E1, E2): Lists of tuples pairing explicit exception types (e.g., `ValueError`) with indices of conditions under which they are raised (from C1/C2), excluding implicit behaviors or unthrown exceptions.

Return Values (R1, R2): Lists of tuples pairing return expressions with condition indices, capturing how return values vary under different conditions.

The 9 types of incompatibility can be detected based on the data structure above. The detection rules are showed in Table x.

Table x

3.4 implementation

We implement the approach using Python, leveraging open-source tools and APIs to balance reproducibility and efficiency. For API extraction, we combine the GitHub API and Python’s standard `ast` library: the GitHub API fetches source code for specific project versions to enable automated data collection, while the `ast` library builds ASTs for function-level slicing—extracting only the function body (excluding imports or global statements outside the API). For LLM interaction, we use DeepSeek-chat API with temperature 0 to minimize randomness and max-tokens 2048 to accommodate structured outputs.

3.5 Multi-Agent

4 Evaluation

4.1 Dataset Construction

Dataset A.

Dataset B.

4.2 Experiment Setup

- **Accuracy Evaluation (RQ1):** How is the overall accuracy of TOOL compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2):** How is the effectiveness of TOOL in different API compatibility types?
- **LLM Generalizability (RQ3):** How is the accuracy of TOOL using different LLMs?
- **Ablation Study (RQ4):**

Baselines. We selected XXX [2], XXX [2], and XXX [2] as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

4.3 Results

4.3.1 Accuracy Evaluation (RQ1).

4.3.2 RQ2.

4.3.3 RQ3.

4.3.4 RQ4.

5 Related Work

5.1 Traditional Compatibility Detection Methods

API compatibility of Python third-party libraries is a critical concern given their widespread use in diverse domains, and relevant detection methods have been a focus of research. SLamothé et al. (2021) conducted a systematic review of API evolution literature, summarizing research trends, state-of-the-art methods, and remaining challenges. For Python, Quan et al. (2022) proposed PyMevol, a static analysis tool that builds a history-sensitive alias-aware API explorer tree to characterize method evolution. Zhang et al. (2024) [2] explored variadic parameter pitfalls in Python, uncovering their potential API compatibility risks. In deep learning, Zhao et al. developed a knowledge-based approach using Stack Overflow-derived knowledge graphs to detect version incompatibilities. For Android, Liu et al. compared nine tools for API-induced compatibility issues, assessing their replicability and effectiveness. Xie et al. addressed cross-repository compatibility with Hera, which builds an offline database to predict, detect, and fix issues across Python and Ubuntu repositories. However, a notable gap exists in these studies: they primarily focus on compatibility issues in manually written code, with limited exploration of how API evolution impacts code generated by automated tools (e.g., LLM-based code completion systems). In this paper, our aim is to address this gap by investigating incompatible API usages in LLM-generated code and developing targeted mitigation strategies.

5.2 LLMs in the Context of API Evolution

Research on LLMs’ reasoning enhancement and practical task adaptation has advanced, with relevance to API evolution-related scenarios. Wei et al. (2022) laid the foundation of Chain-of-Thought (CoT) prompting, enabling LLMs to conduct multi-step reasoning—critical for handling complex logic in API evolution. Li et al. (2025) proposed Structured CoT (SCoT) prompting, integrating sequential, branch, and loop structures to boost code generation accuracy, aiding code adaptation in evolving environments. Zhang et al. (2025) optimized CoT with reasoning patterns (Pattern-CoT), selecting diverse demonstrations to enhance LLMs’ robustness in varied reasoning scenarios. Liu et al. (2024) presented ERA-CoT, using entity relationship analysis to strengthen LLMs’ understanding of relational changes, useful for tracking associated modifications in evolution. For vulnerability detection linked to evolution, Ma et al. (2024) developed iAudit (combining fine-tuning and LLM agents) for smart contract auditing, and Sun et al. (2024) proposed GPTScan (fusing GPT with static analysis) for logic vulnerability detection. Stechly et al. (2024) noted CoT’s limitation in planning—reliance on task-specific prompts, which affects adaptability in dynamic evolution scenarios. Notably, none design prompts for API compatibility or prioritize Python APIs. We address this by crafting API compatibility-focused prompts (integrating Python API version traits/parameter changes) and optimizing for Python’s dynamic features (e.g., variadic parameters) to align LLMs with Python API evolution.

References

- [1] Vahid Garousi and Mika V Mäntylä. 2016. A systematic literature review of literature reviews in software testing. *Information and Software Technology* 80 (2016), 195–216.
- [2] Zaixi Zhang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2022. Fldetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2545–2555.